Controlling Backtracking

Say you're given the problem of searching a string for a sequence of *X* characters not immediately followed by an *O*. You know that a string of *X_s can be represented as* X+, *and you can use a lookahead to check that it isn't followed by an _O*, so you code up the pattern /(X+)(?!O)/. Let's try it.

This works:

```
re = /(X+)(?!O)/
re.match("test XXXY")[0] # => "XXX"
```

But, unfortunately, so does this, though with a slightly different match:

```
re = /(X+)(?!O)/
re.match("test XXXO")[0] # => "XX"
```

Why did the second match succeed? Well, the regular expression engine saw the X+ in the pattern and happily gobbled up all the Xs in the string. It then saw the pattern (?!O), saying that it should not now be looking at an O. Unfortunately, it is looking at an O, so the match doesn't succeed. But the engine doesn't give up. No sir! Instead it says, "Maybe I was wrong to consume every single X in the string. Let's try consuming one less and see what happens." This is called *backtracking*—when a match fails, the engine goes back and tries to match a different way. In this case, by backtracking past a single character, it now finds itself looking at the last X in the string (the one before the final O). And that X is not an O, so the negative lookahead succeeds, and the pattern matches. Look at the output of the previous program: there are three _X_s in the first match but only two in the second.

But this wasn't the intent of our regexp. Once it finds a sequence of Xs, those Xs should be considered as a unit. We don't want the sequence to split to all but one of them, with the last of them then being the terminator of the pattern. We can get that behavior by telling Ruby not to backtrack once it finds a string of Xs. There are a couple of ways of doing this.

The sequence (?>EXPRESSION) nests an independent regular expression within the first regular expression. This expression is anchored at the current match position. If it consumes characters, these will no longer be available to the higher-level regular expression. This construct, called *atomic grouping* therefore inhibits backtracking.

Let's try it with our previous code, making the set of Xs an atomic grouping.

This one still works:

```
re = /((?>X+))(?!O)/
re.match("test XXXY")[0] # => "XXX"
```

But now this one doesn't:

```
re = /((?>X+))(?!O)/
re.match?("test XXXO") # => false
```

And this finds the second string of Xs:

```
re = /((?>X+))(?!O)/
re.match("test XXXO XXXXY")[0] # => "XXXX"
```